

P4747 [CERC2017] Intrinsic Interval

性质 1: 如果 $[l1, r1]$ 是连续段, $[l2, r2]$ 是连续段, 且这两个区间相交但不包含, 则 $[l1, r1] \cap [l2, r2]$ 、 $[l1, r1] \cup [l2, r2]$ 均为连续段。

性质 2: 包含 $[l, r]$ 的最短连续段 $[a, b]$ 满足其他所有包含 $[l, r]$ 的连续段 $[c, d]$ 都包含 $[a, b]$ 。(否则 $[a, b] \cap [c, d]$ 仍符合要求且更优)

那么, 只需要找到包含 $[l, r]$ 的连续段中右端点的最小值和左端点的最大值即可。可以使用线段树求出以每个点为右端点, 构成连续段的最小左端点, 以及以每个点为左端点, 构成连续段的最大右端点。查询就直接扫描线树状数组, 总复杂度: $O(n \log n + m \log n)$ 。

AGC061D Almost Multiplication Table

二分答案, 判定是否存在一组解答案 $\leq D$ 。不妨令解的 $x_n \leq y_m$, $M = 10^9$ 。

我们初始令 $x_i = i, y_i = i + D + M$, 之后:

- 如果 $x_i y_j$ 过大, 就调整 y_j 。
- 如果 $x_i y_j$ 过小, 就调整 x_i 。
- 如果不满足 $x_i < x_{i+1}$, 就调整 x_{i+1} 。
- 如果不满足 $y_i < y_{i+1}$, 就调整 y_i 。

其核心是 x_i 永远是第 i 行合法 x_i 的下界, y_i 永远是第 i 列合法 y_i 的上界。

因为 $x_n \leq \sqrt{D + M}$, 所以对 x 的调整次数不超过 $O(n\sqrt{M})$ 次。而每一次调整 x , 对 y 调整的复杂度为 $O(m)$ 。因此再结合外层的二分答案, 总复杂度为 $O(nm\sqrt{M} \log M)$ 。

ARC184B 123 Set

我们考虑将 $1 \sim n$ 的数按照去掉 2、3 质因子后剩下的数值分类。

显然, 每个类互相独立, 即: 我们可以独立计算每类数至少花多少次操作能全覆盖, 再加起来就是答案。

然后考虑 sz 相同的类显然等价, 因此, 我们只需要计算 $O(\log_2 n * \log_3 n)$ 种类的答案即可。

这不好使用贪心或网络流计算, 但是考虑到 $2^{\log_3 n}$ 比较小, 因此使用轮廓线状压 dp 来实现即可。

QOJ4217 Graph Coloring

本题构造的三种路径:

- 位运算构造: 有全序关系的图直接 x 与 y 的权值为 $highbit(x \oplus y)$ 即可。但是现在是没有全序关系的图, 我们考虑先往左再往右的情况会出现问题, 因此我们将往左的边的权值变成 $highbit(x \oplus y) + 12$ 即可, 需要使用 24 种颜色。
- 分治构造: 有全序关系的图我们将点分成左右两半。不可能既在左边走一步, 又在右边走一步, 也不可能跨左右走两步。因此, 我们将左边到右边的边连上一种相同的权值, 然后递归到左右分别连边即可。没有全序关系的图我们只需要左边到右边连一种颜色, 右边到左边连另一种颜色, 然后递归到左右分别连边即可。需要使用 24 种颜色。

- 转换题意之后集合型构造：我们把问题转换为对每个点分配一个颜色集合 S_x ，表示这个点往外连的边的颜色集合为 S_x ，则对于一个边 $x \rightarrow y$ ，其颜色必须 $\in S_x$ 且 $\notin S_y$ 。因此我们需要满足对每个边 $x \rightarrow y$ ，都不能有 $S_x \subseteq S_y$ 。直接把所有 14 位二进制数中 $popcnt = 7$ 的拿出来给 S_i 分配互不相同的 n 个即可。需要使用 14 种颜色。

QOJ4218 Hidden Graph

图的任一生成子图都可以划分成 $k + 1$ 个独立集，因此我们不断加入点，维护目前已加入点构成的生成子图以及这个子图构成的 $k + 1$ 个独立集。（每次加入一个点之后重构出这 $k + 1$ 个独立集）

一个点插入的时候可以直接暴力查询与 $k + 1$ 个独立集之间的连边情况，每个点最多会浪费 $k + 1$ 次查询。

因此，总询问次数 $\leq n(k + 1) + m \leq 2nk + n$ ，可以通过本题。

P5115 Check,Check,Check one two!

方法一：考虑用组合意义拆掉式子。枚举 i, j ，再枚举 pre 和 suf ，当 $pre = k1 + 1$ 时，贡献 $-k1$ ，当 $pre \leq k1$ 时，贡献 1，其余贡献 0；当 $suf = k2 + 1$ 时，贡献 $-k2$ ，当 $suf \leq k2$ 时，贡献 1，其余贡献 0，总贡献为 pre 的贡献与 suf 的贡献的积。变为枚举 i, j, len, pos ，其中 $s[i, i + len - 1] = s[j, j + len - 1]$ 。所有 pos 的贡献和只与 len 有关，因此只需要计算有多少个长度为 len 的两个字符串相等，以及长度为 len 的 pos 贡献和。两者都是平凡的，复杂度 $O(n \log n)$ ，瓶颈在建 SA。

方法二：我们把 $\langle i, j, lcp, lcs \rangle$ 在对应 $i - lcs + 1, j - lcs + 1$ 位置统计，令其分别为 p, q ，需要满足： $s_p = s_q, s_{p-1} \neq s_{q-1}$ 。接下来考虑往后接的最大长度 $len = lcp(p, q)$ ，那么满足 $i - lcs + 1 = p, j - lcs + 1 = q$ 的所有 $\langle i, j, lcp, lcs \rangle$ 的贡献和就只与 len 有关了，可以表示为 $f(len)$ 。于是现在只需要 $\forall i, j, s_{i-1} \neq s_{j-1} : g(lcp(i, j)) ++$ ，求最终的 $g()$ 。对于 $s_{i-1} \neq s_{j-1}$ 的限制，直接用整体减去相等的，整体的可以使用 SA+并查集/笛卡尔树/单调栈，在有 m 个后缀的情况下复杂度为 $O(m)$ 。而“相等”的所有情况的后缀总数是 n ，“整体”的后缀个数也是 n ，因此可以线性计算出 $g()$ ，复杂度 $O(n \log n)$ ，瓶颈在建 SA。

方法三：我们建立正反串 SA 的笛卡尔树，那么 $lcp(i, j)$ 和 $lcs(i, j)$ 实际上就分别是第一个树和第二个树上 i, j 的 lca 。考虑在第一棵笛卡尔树上启发式合并，在第二棵树上使用 [P4211 \[LNOI2014\] LCA](#) 的技巧，把贡献在树链上差分，之后使用树剖 + 线段树/BIT 维护可以做到 $O(n \log^3 n)$ 。将树剖变为全局平衡二叉树可以做到 $O(n \log^2 n)$ 。

方法四：我们沿用方法三，不过把“在第一棵树上启发式合并”变为“在第一棵树上边分治”，但第一棵树上的贡献为 $f(lca(i, j))$ ，与树分治并不兼容，需要把“ lca 信息”变为“路径信息”，考虑在方法三中我们使用差分变为了 $lca(i, j)$ 到根的路径上所有点权的和，我们现在考虑把贡献差分放到边上，根的贡献不放到任何地方，那么 $f(lca(i, j)) = \frac{dist(1, i) + dist(1, j) - dist(i, j)}{2} + v_1$ 。现在的贡献不含 lca 信息了，套用边分治后， $f(lca(i, j))$ 就可以拆为 $g(i) + h(j)$ 的形式。我们对当前分治层的所有点，建立第二棵树上的虚树，之后问题变为，给定一些 i ，查询 j 在第二棵树上 $d(lca(i, j)) * (g(i) + h(j))$ 的和，可以搜一遍整颗虚树线性求出。复杂度 $O(n \log^2 n)$ ，瓶颈在建虚树。